

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

1. Introduction

Deep convolutional neural networks [22, 21] have led to a series of breakthroughs for image classification [21, 50, 40]. Deep networks naturally integrate low/mid/high-level features [50] and classifiers in an end-to-end multi-layer fashion, and the “levels” of features can be enriched by the number of stacked layers (depth). Recent evidence [41, 44] reveals that network depth is of crucial importance, and the leading results [41, 44, 13, 16] on the challenging ImageNet dataset [36] all exploit “very deep” [41] models, with a depth of sixteen [41] to thirty [16]. Many other non-trivial visual recognition tasks [8, 12, 7, 32, 27] have also

¹<http://image-net.org/challenges/LSVRC/2015/> and <http://mscoco.org/dataset/#detections-challenge2015>.

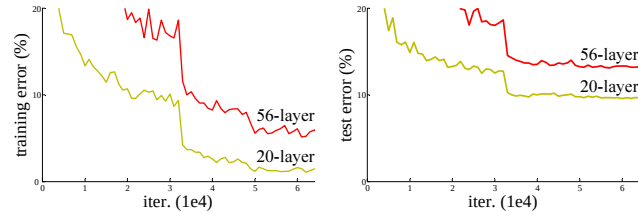


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [1, 9], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 9, 37, 13] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with back-propagation [22].

When deeper networks are able to start converging, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. Unexpectedly, such degradation is *not caused by overfitting*, and adding more layers to a suitably deep model leads to *higher training error*, as reported in [11, 42] and thoroughly verified by our experiments. Fig. 1 shows a typical example.

The degradation (of training accuracy) indicates that not all systems are similarly easy to optimize. Let us consider a shallower architecture and its deeper counterpart that adds more layers onto it. There exists a solution *by construction* to the deeper model: the added layers are *identity* mapping, and the other layers are copied from the learned shallower model. The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. But experiments show that our current solvers on hand are unable to find solutions that

图像识别的深度残差学习

何恺明 张翔 任少卿 孙剑微软研究院{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

摘要

更深的神经网络往往更难训练。我们提出了一种残差学习框架，以简化那些远比此前使用的网络更深得多的网络训练过程。我们显式地将各层重构为参考层输入学习的残差函数，而非学习无参考的函数。我们提供了充分的实验证据，证明这些残差网络更容易优化，并且能够从大幅增加的深度中提升精度。在 ImageNet 数据集上我们评估了深度最高达 152 层的残差网络——8×比 VGG 网络更深 [41] 但复杂度仍较低。此类残差网络的集成模型在 ImageNet 测试集上实现了 3.57% 的错误率。该成果在 ILSVRC 2015 分类任务中位列第一。我们还呈现了在 100 层与 1000 层 CIFAR-10 数据集上的分析结果。

表示的深度对于许多视觉识别任务至关重要。仅凭借我们极深的表示，我们在 COCO 目标检测数据集上获得了 28% 的相对提升。深度残差网络是我们参与 ILSVRC & COCO 2015 竞赛作品的基石¹，我们还在 ImageNet 检测、ImageNet 定位、COCO 检测以及 COCO 分割任务中均荣获第一。

1. 引言

深度卷积神经网络 [22, 21] 已在图像分类 [21, 50, 40] 领域取得了一系列突破性进展。深层网络以端到端的多层形式自然地低/中/高级特征 [50] 与分类器相结合，且特征的“层级”可通过堆叠的层数（深度）得以丰富。最新证据[41, 44] 表明，网络深度至关重要，在极具挑战性的 ImageNet 数据集 [36] 上取得的领先成果 [41, 44, 13, 16] 均采用了“极深” [41] 模型，其深度通常为十六至三十层 [41] [16]。许多其他非平凡的视觉识别任务 [8, 12, 7, 32, 27] 也已

¹<http://image-net.org/challenges/LSVRC/2015/> 和 <http://mscoco.org/dataset/#detections-challenge2015>。

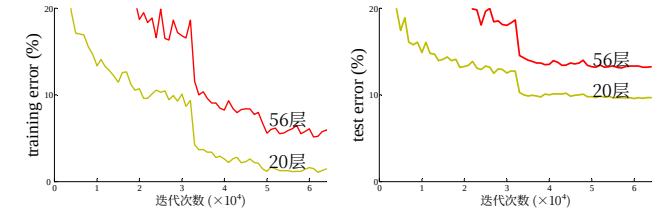


图1. CIFAR-10数据集上20层与56层“普通”网络的训练误差（左）和测试误差（右）。更深的网络具有更高的训练误差，因而测试误差也更高。ImageNet上的类似现象如图4所示。

极大地受益于极深模型。

出于对深度的重要性的考量，一个自然的问题是：构建更好的网络是否仅仅像堆叠更多层那样简单？解答这一问题的障碍之一是臭名昭著的梯度消失/爆炸问题 [1, 9]，它从训练初期便阻碍了模型收敛。然而，借助归一化初始化 [23, 9, 37, 13] 与中间归一化层[16]，该问题已在很大程度上得到缓解，使得拥有数十层的网络在使用反向传播配合随机梯度下降（SGD）训练时，能够顺利开启收敛过程 [22]。

当更深的网络能够开始收敛时，一个退化问题便被暴露出来：随着网络深度的增加，准确率会逐渐饱和（这或许不足为奇），随后迅速下降。出乎意料的是，这种退化是并非由过拟合引起的，且在适当深度的模型上追加更多层会导致更高的训练误差，正如 [11, 42] 所述并经我们实验充分验证。图 1 展示了一个典型示例。

训练准确率（的下降）表明，并非所有系统都同样易于优化。让我们考虑一个较浅的网络架构及其更深的对应模型，后者在其基础上增加了更多层。对于该更深模型，存在一种通过构造得到的解 通过构造：新增的层为恒等映射，其余层则从已学习的较浅模型中直接复制而来。这种构造解的存在表明，更深模型的训练误差不应高于其较浅的对应模型。但实验表明，我们手头的现有求解器无法找到那些

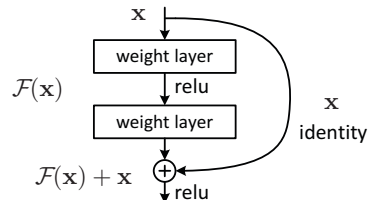


Figure 2. Residual learning: a building block.

are comparably good or better than the constructed solution (or unable to do so in feasible time).

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. To the extreme, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers.

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with “shortcut connections” (Fig. 2). Shortcut connections [2, 34, 49] are those skipping one or more layers. In our case, the shortcut connections simply perform *identity* mapping, and their outputs are added to the outputs of the stacked layers (Fig. 2). Identity shortcut connections add neither extra parameter nor computational complexity. The entire network can still be trained end-to-end by SGD with backpropagation, and can be easily implemented using common libraries (*e.g.*, Caffe [19]) without modifying the solvers.

We present comprehensive experiments on ImageNet [36] to show the degradation problem and evaluate our method. We show that: 1) Our extremely deep residual nets are easy to optimize, but the counterpart “plain” nets (that simply stack layers) exhibit higher training error when the depth increases; 2) Our deep residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks.

Similar phenomena are also shown on the CIFAR-10 set [20], suggesting that the optimization difficulties and the effects of our method are not just akin to a particular dataset. We present successfully trained models on this dataset with over 100 layers, and explore models with over 1000 layers.

On the ImageNet classification dataset [36], we obtain excellent results by extremely deep residual nets. Our 152-layer residual net is the deepest network ever presented on ImageNet, while still having lower complexity than VGG nets [41]. Our ensemble has **3.57%** top-5 error on the

ImageNet *test* set, and won the *1st place in the ILSVRC 2015 classification competition*. The extremely deep representations also have excellent generalization performance on other recognition tasks, and lead us to further *win the 1st places on: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation* in ILSVRC & COCO 2015 competitions. This strong evidence shows that the residual learning principle is generic, and we expect that it is applicable in other vision and non-vision problems.

2. Related Work

Residual Representations. In image recognition, VLAD [18] is a representation that encodes by the residual vectors with respect to a dictionary, and Fisher Vector [30] can be formulated as a probabilistic version [18] of VLAD. Both of them are powerful shallow representations for image retrieval and classification [4, 48]. For vector quantization, encoding residual vectors [17] is shown to be more effective than encoding original vectors.

In low-level vision and computer graphics, for solving Partial Differential Equations (PDEs), the widely used Multigrid method [3] reformulates the system as subproblems at multiple scales, where each subproblem is responsible for the residual solution between a coarser and a finer scale. An alternative to Multigrid is hierarchical basis preconditioning [45, 46], which relies on variables that represent residual vectors between two scales. It has been shown [3, 45, 46] that these solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions. These methods suggest that a good reformulation or preconditioning can simplify the optimization.

Shortcut Connections. Practices and theories that lead to shortcut connections [2, 34, 49] have been studied for a long time. An early practice of training multi-layer perceptrons (MLPs) is to add a linear layer connected from the network input to the output [34, 49]. In [44, 24], a few intermediate layers are directly connected to auxiliary classifiers for addressing vanishing/exploding gradients. The papers of [39, 38, 31, 47] propose methods for centering layer responses, gradients, and propagated errors, implemented by shortcut connections. In [44], an “inception” layer is composed of a shortcut branch and a few deeper branches.

Concurrent with our work, “highway networks” [42, 43] present shortcut connections with gating functions [15]. These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, high-

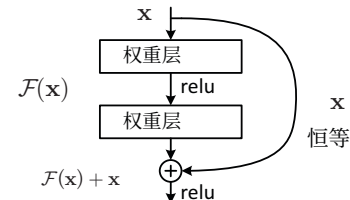


图2. 残差学习：一个基础模块。

与上述构造解效果相当甚至更优的解（或在合理时间内无法实现）。

在本文中，我们通过引入深度残差学习框架来解决退化问题。与期望少数堆叠层直接拟合目标底层映射不同，我们显式地让这些层去拟合残差映射。形式上，将期望的底层映射记为 $\mathcal{H}(\mathbf{x})$ ，我们令堆叠的非线性层拟合 $\mathcal{F}(\mathbf{x})$ 的另一个映射： $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。原始映射被重构为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。我们假设优化残差映射比优化原始且缺乏参考的映射更为容易。极端情况下，若恒等映射即为最优解，则直接将残差归零显然比使用堆叠的非线性层去拟合恒等映射要简单得多。

$\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 的表达式可通过带有“快捷连接”的前馈神经网络来实现（图2）。快捷连接 [2, 34, 49] 是指跳过一层或多层的连接。在本例中，快捷连接仅执行恒等映射，并将其输出与堆叠层的输出相加（图2）。恒等快捷连接既不引入额外参数，也不增加计算复杂度。整个网络仍可通过带反向传播的随机梯度下降法（SGD）进行端到端训练，并能借助常用库（例如，Caffe [19]）轻松实现，且无需修改求解器。

我们在 ImageNet[36] 上开展了综合实验，旨在揭示退化现象并验证所提方法的有效性。实验表明：1) 我们所构建的极深残差网络易于优化，而对比的“普通”网络（仅简单堆叠各层）随着深度增加会出现更高的训练误差；2) 我们的深残差网络能够轻松从大幅增加的深度中获取精度收益，其结果显著优于以往各类网络。

在 CIFAR-10 数据集上也观察到了类似的现象[20]，这表明我们方法所面临的优化困难及其效果并非仅针对某一特定数据集。我们在此数据集上成功训练了超过 100 层的模型，并探索了具有超过 1000 层的模型。

在 ImageNet 分类数据集 [36] 上，我们取得了极深残差网络的优异成果。我们的 152-层残差网络是 ImageNet 上呈现过的最深网络，且其复杂度仍低于 VGG 网络 [41]。我们的集成模型拥有 **3.57%** 的 top-5 错误率，在

ImageNet 测试集，并在 ILSVRC2015 分类竞赛中获得第一名。这些极深的表示在其他识别任务上也展现出优异的泛化性能，促使我们在 ILSVRC 与 COCO 2015 竞赛中进一步获得包括 ImageNet 检测、ImageNet 定位、COCO 检测与 COCO 分割在内的多项冠军。这一充分证据表明，残差学习原理具有通用性，我们预期它同样适用于其他视觉与非视觉领域的问题。

2. 相关工作

残差表示。 在图像识别中，VLAD[18]是一种通过相对于字典的残差向量进行编码的表示方法，而Fisher向量 [30] 可被表述为VLAD的概率化版本 [18]。两者均为图像检索与分类的强大浅层表示方法 [4, 48]。对于向量量化，已有研究证明对残差向量进行编码比对原始向量进行编码更为有效 [17]。

在低层视觉与计算机图形学中，为解决偏微分方程（PDEs），广泛使用的多网格法 [3] 将系统重构为多尺度子问题，其中每个子问题负责求解较粗尺度与较细尺度间的残差解。多网格法的替代方案之一是层次基预处理 [45, 46]，其依赖于表示两尺度间残差向量的变量。已有研究证实[3, 45, 46]，这些求解器的收敛速度远快于未考虑解的残差特性的标准求解器。这些方法表明，良好的问题重构或预处理能够有效简化优化过程。

捷径连接。 促成捷径连接的实践与理论 [2, 34, 49] 长期以来一直受到广泛研究。训练多层感知机（MLP）的早期做法之一是添加一个从网络输入端连接到输出端 [34, 49] 的线性层。在 [44, 24] 中，部分中间层直接与辅助分类器相连，旨在缓解梯度消失与爆炸问题。[39, 38, 31, 47] 所著文献提出了利用捷径连接来对层响应、梯度及传播误差进行中心化的方法。在 [44] 中，“Inception”层由一条捷径分支与若干更深层次的分支组合而成。

与此同时，“高速网络” [42, 43] 引入了带有门控函数的跳跃连接 [15]。这些门控函数依赖于数据且包含参数，与我们的无参数恒等捷径形成对比。当门控跳跃连接“关闭”（趋近于零）时，高速网络中的各层表示的是非残差函数。相反，我们的方法始终学习残差函数；我们的恒等捷径永远不会被关闭，所有信息均始终得以传递，仅需额外学习残差函数。此外，高-

way networks have not demonstrated accuracy gains with extremely increased depth (*e.g.*, over 100 layers).

3. Deep Residual Learning

3.1. Residual Learning

Let us consider $\mathcal{H}(\mathbf{x})$ as an underlying mapping to be fit by a few stacked layers (not necessarily the entire net), with $\mathbf{x}$ denoting the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions², then it is equivalent to hypothesize that they can asymptotically approximate the residual functions, *i.e.*, $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming that the input and output are of the same dimensions). So rather than expect stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let these layers approximate a residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original function thus becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to asymptotically approximate the desired functions (as hypothesized), the ease of learning might be different.

This reformulation is motivated by the counterintuitive phenomena about the degradation problem (Fig. 1, left). As we discussed in the introduction, if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that the solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings.

In real cases, it is unlikely that identity mappings are optimal, but our reformulation may help to precondition the problem. If the optimal function is closer to an identity mapping than to a zero mapping, it should be easier for the solver to find the perturbations with reference to an identity mapping, than to learn the function as a new one. We show by experiments (Fig. 7) that the learned residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning.

3.2. Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is shown in Fig. 2. Formally, in this paper we consider a building block defined as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

Here $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping to be learned. For the example in Fig. 2 that has two layers, $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ in which σ denotes

ReLU [29] and the biases are omitted for simplifying notations. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and element-wise addition. We adopt the second nonlinearity after the addition (*i.e.*, $\sigma(\mathbf{y})$, see Fig. 2).

The shortcut connections in Eqn.(1) introduce neither extra parameter nor computation complexity. This is not only attractive in practice but also important in our comparisons between plain and residual networks. We can fairly compare plain/residual networks that simultaneously have the same number of parameters, depth, width, and computational cost (except for the negligible element-wise addition).

The dimensions of $\mathbf{x}$ and $\mathcal{F}$ must be equal in Eqn.(1). If this is not the case (*e.g.*, when changing the input/output channels), we can perform a linear projection W_s by the shortcut connections to match the dimensions:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

We can also use a square matrix W_s in Eqn.(1). But we will show by experiments that the identity mapping is sufficient for addressing the degradation problem and is economical, and thus W_s is only used when matching dimensions.

The form of the residual function $\mathcal{F}$ is flexible. Experiments in this paper involve a function $\mathcal{F}$ that has two or three layers (Fig. 5), while more layers are possible. But if $\mathcal{F}$ has only a single layer, Eqn.(1) is similar to a linear layer: $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$, for which we have not observed advantages.

We also note that although the above notations are about fully-connected layers for simplicity, they are applicable to convolutional layers. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ can represent multiple convolutional layers. The element-wise addition is performed on two feature maps, channel by channel.

3.3. Network Architectures

We have tested various plain/residual nets, and have observed consistent phenomena. To provide instances for discussion, we describe two models for ImageNet as follows.

Plain Network. Our plain baselines (Fig. 3, middle) are mainly inspired by the philosophy of VGG nets [41] (Fig. 3, left). The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. The total number of weighted layers is 34 in Fig. 3 (middle).

It is worth noticing that our model has *fewer* filters and *lower* complexity than VGG nets [41] (Fig. 3, left). Our 34-layer baseline has 3.6 billion FLOPs (multiply-adds), which is only 18% of VGG-19 (19.6 billion FLOPs).

速网络在深度极大幅度增加的情况下（例如，超过100层），并未展现出准确率上的提升。

3. 深度残差学习

3.1. 残差学习

我们将 $\mathcal{H}(\mathbf{x})$ 视为一个待拟合的底层映射，需由少量堆叠层（未必是整个网络）来拟合，其中 $\mathbf{x}$ 表示这些层中第一层的输入。若假设多个非线性层能够渐近地逼近复杂函数²，则等价于假设它们能够渐近地逼近残差函数，即， $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ （假设输入与输出具有相同维度）。因此，我们不期望堆叠层去逼近 $\mathcal{H}(\mathbf{x})$ ，而是显式地令这些层去逼近残差函数 $\mathcal{F}(\mathbf{x})$: $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。原始函数由此变为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。尽管这两种形式均能渐近地逼近目标函数（如前述假设），但它们的学习难易程度可能会有所不同。

这种重构的动机来源于退化问题（图1左侧）的反直觉现象。正如我们在引言中所讨论的，如果新增的层能够被构建为恒等映射，那么更深的模型其训练误差不应高于较浅的对应模型。退化问题表明，求解器在利用多个非线性层逼近恒等映射时可能会遇到障碍。采用残差学习重构后，若恒等映射为最优解，求解器只需将多个非线性层的权重驱动至趋近于零，即可实现对该映射的逼近。

在实际场景下，恒等映射不太可能是最优的，但我们的公式改写可能有助于对该问题进行预处理。如果最优函数更接近恒等映射而非零映射，那么求解器基于恒等映射寻找扰动，肯定比将其作为新函数直接学习要容易得多。我们通过实验（图7）发现，所学习的残差函数通常响应较小，这表明恒等映射能够提供合理的预处理。

3.2. 通过捷径连接实现恒等映射

我们将残差学习应用于每隔几层的堆叠结构。如图2所示为该基本模块。形式上，本文考虑的模块定义如下：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

此处 $\mathbf{x}$ 与 $\mathbf{y}$ 分别为所考虑层级的输入与输出向量。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 代表需要学习的残差映射。在图2所示的两层示例中， $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ 中的 σ 表示

²然而，这一假设仍然是一个开放性问题。详见 [28]。

为简化符号表示，省略了 ReLU [29]及偏置项。操作 $\mathcal{F} + \mathbf{x}$ 由跳跃连接和逐元素相加来实现。我们在相加之后采用第二个非线性函数（即， $\sigma(\mathbf{y})$ ，见图2）。

式(1)中的捷径连接既不会引入额外参数，也不会增加计算复杂度。这不仅在实际应用中十分诱人，也在普通网络与残差网络的对比中具有重要意义。我们可以对普通网络与残差网络进行公平的比较，使它们同时具备相同的参数量、深度、宽度以及计算开销（可忽略的逐元素加法除外）。

在公式(1)中， $\mathbf{x}$ 与 $\mathcal{F}$ 的维度必须相等。若非如此（例如，在更改输入/输出通道时），我们可以借助捷径连接执行线性投影 W_s 以匹配维度：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

我们也可以在公式(1)中引入方阵 W_s 。但实验将证明，恒等映射足以应对退化问题且更加经济，因此 W_s 仅在需要对齐维度时使用。

残差函数 $\mathcal{F}$ 的形式十分灵活。本文实验采用的函数 $\mathcal{F}$ 包含两至三层（图5），理论上也可增加层数。但如果 $\mathcal{F}$ 仅包含单层结构，则公式(1)退化为类似线性层的结构： $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$ ，对此我们并未观察到显著优势。

我们还需指出，尽管上述记号为简洁起见主要针对全连接层，但它们完全适用于卷积层。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 可表示多个卷积层。逐元素相加操作将在两个特征图上按通道逐一进行。

3.3. 网络架构

我们对多种普通网络与残差网络进行了测试，并观察到了规律性的现象。为便于讨论，以下将介绍两个用于 ImageNet 的模型。

普通网络。 我们的普通基线模型（图3中）主要受 VGG网络 [41]（图3左）设计理念的启发。卷积层大多包含 3×3 个滤波器，并遵循两项简单的设计规则：(i) 对于相同的输出特征图尺寸，各层具有相同数量的滤波器；以及(ii) 若特征图尺寸减半，则将滤波器数量翻倍，以维持每层的计算复杂度。我们通过步幅为2的卷积层直接进行下采样。该网络以全局平均池化层和含Softmax的1000类全连接层结尾。图3（中）所示网络的带权重的层总数为34层。

值得注意的是，与VGG网络 [41]相比，我们的模型具有更少的滤波器以及更低的复杂度，位于图3左侧。我们的34层基线模型的浮点运算次数（乘法操作）为36亿，其仅为VGG-19的18%（196亿FLOPs）。

²This hypothesis, however, is still an open question. See [28].

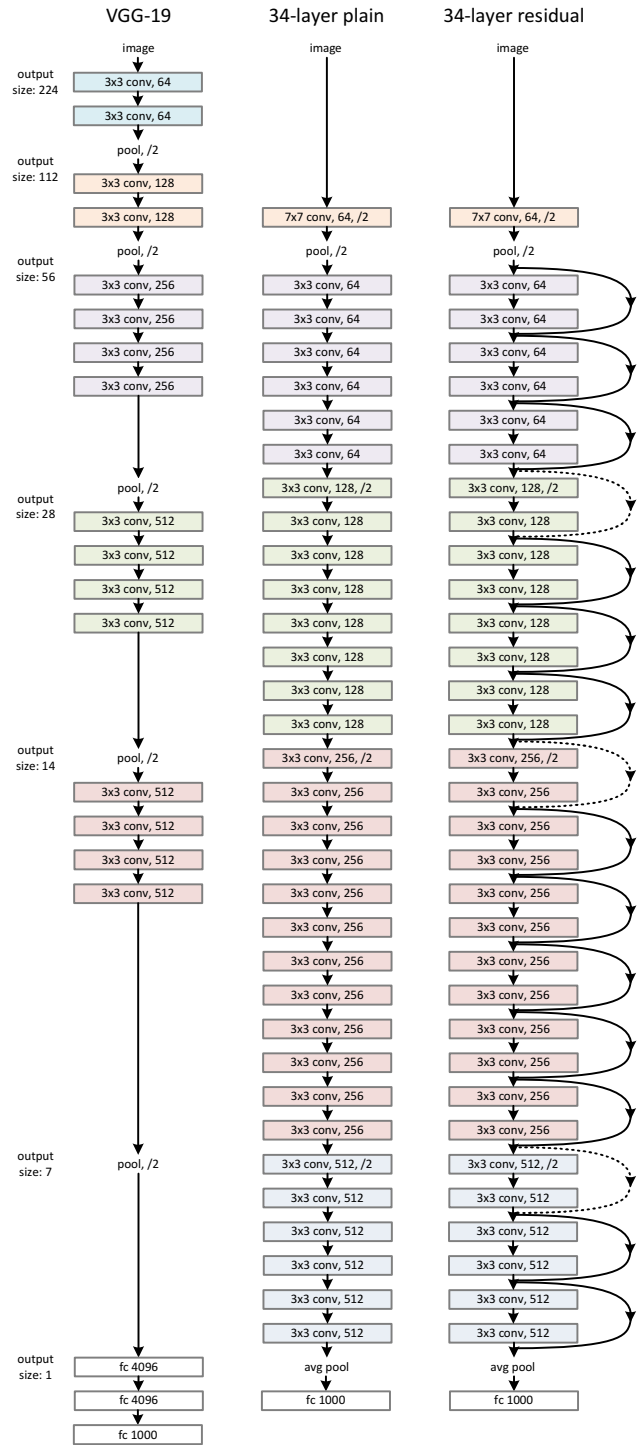


Figure 3. Example network architectures for ImageNet. **Left:** the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle:** a plain network with 34 parameter layers (3.6 billion FLOPs). **Right:** a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Residual Network. Based on the above plain network, we insert shortcut connections (Fig. 3, right) which turn the network into its counterpart residual version. The identity shortcuts (Eqn.(1)) can be directly used when the input and output are of the same dimensions (solid line shortcuts in Fig. 3). When the dimensions increase (dotted line shortcuts in Fig. 3), we consider two options: (A) The shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions. This option introduces no extra parameter; (B) The projection shortcut in Eqn.(2) is used to match dimensions (done by 1×1 convolutions). For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2.

3.4. Implementation

Our implementation for ImageNet follows the practice in [21, 41]. The image is resized with its shorter side randomly sampled in [256, 480] for scale augmentation [41]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [13] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [14], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21]. For best results, we adopt the fully-convolutional form as in [41, 13], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).

4. Experiments

4.1. ImageNet Classification

We evaluate our method on the ImageNet 2012 classification dataset [36] that consists of 1000 classes. The models are trained on the 1.28 million training images, and evaluated on the 50k validation images. We also obtain a final result on the 100k test images, reported by the test server. We evaluate both top-1 and top-5 error rates.

Plain Networks. We first evaluate 18-layer and 34-layer plain nets. The 34-layer plain net is in Fig. 3 (middle). The 18-layer plain net is of a similar form. See Table 1 for detailed architectures.

The results in Table 2 show that the deeper 34-layer plain net has higher validation error than the shallower 18-layer plain net. To reveal the reasons, in Fig. 4 (left) we compare their training/validation errors during the training procedure. We have observed the degradation problem - the

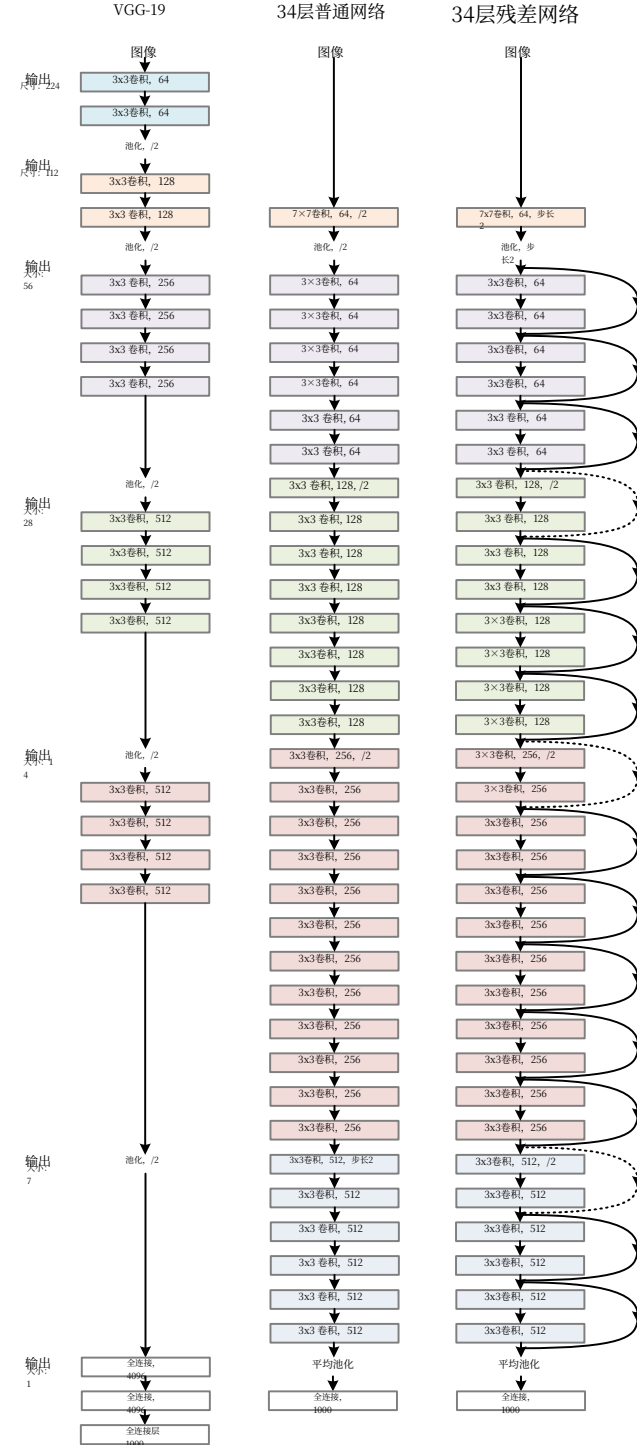


图3. ImageNet 示例网络架构。左侧：以 VGG-19 模型 [41] (196 亿 FLOPs) 为参考。中间：包含 34 个参数层的普通网络 (36 亿 FLOPs)。右侧：包含 34 个参数层的残差网络 (36 亿 FLOPs)。虚线捷径用于增加维度。表1 展示了更多细节及其他变体。

残差网络。 基于上述普通网络，我们插入跳跃连接（图3右侧），将网络转换为其对应的残差版本。恒等映射跳跃连接（公式(1)）在输入和输出维度相同时可直接使用（图3中的实线跳跃连接）。当维度增加时（虚线跳跃连接见图3），我们考虑两种方案：（A）跳跃连接仍执行恒等映射，通过填充额外的零值来适配增加的维度。此方案不引入额外参数；（B）使用公式(2)中的投影跳跃连接来匹配维度（由 1×1 卷积实现）。对于这两种方案，当跳跃连接跨越两种不同尺寸的特征图时，均设置步长为2。

3.4. 实现

我们在 ImageNet 上的实现遵循 [21, 41] 中的做法。为进行尺度增强 [41]，图像短边在 [256, 480] 范围内随机采样并调整尺寸。从图像或其水平翻转图中随机裁剪 224×224 区域，并减去像素均值 [21]。采用 [21] 中的标准颜色增强。遵循 [16] 的做法，我们在每个卷积之后、激活之前引入批量归一化（BN）[16]。权重初始化方法同 [13]，所有普通/残差网络均从头训练。优化器使用 SGD，最小批次大小设为 256。学习率从 0.1 开始，当误差趋于平稳时除以 10，模型最多训练 60×10^4 次迭代。权重衰减为 0.0001，动量为 0.9。遵循 [16] 的做法，我们不使用 Dropout [14]。

在测试阶段，为了进行对比研究，我们采用标准的 10-crop 测试方法 [21]。为获得最佳结果，我们采用如 [41, 13] 所示的全卷积形式，并在多个尺度上对预测分数求平均（图像被缩放至短边尺寸为 $\{224, 256, 384, 480, 640\}$ ）。

4. 实验

4.1. ImageNet 分类

我们在包含 1000 个类别的 ImageNet 2012 分类数据集 [36] 上评估我们的方法。模型使用 128 万张训练图像进行训练，并在 5 万张验证图像上进行评估。我们还获取了由测试服务器报告的 10 万张测试图像的最终结果。我们同时评估 top-1 和 top-5 错误率。

普通网络。 我们先评估 18 层和 34 层的普通网络。34 层普通网络如图 3（中间）所示。18 层普通网络的结构类似。详细架构请参见表 1。

表 2 的结果表明，更深的 34 层普通网络的验证误差高于较浅的 18 层普通网络。为揭示其原因，我们在图 4（左）中比较了它们在训练过程中的训练/验证误差。我们观察到了退化问题——该

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

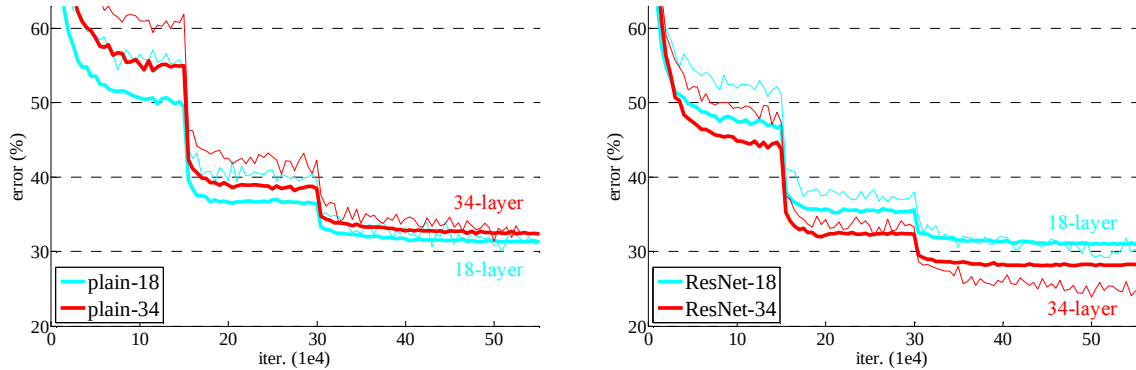


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

34-layer plain net has higher *training* error throughout the whole training procedure, even though the solution space of the 18-layer plain network is a subspace of that of the 34-layer one.

We argue that this optimization difficulty is *unlikely* to be caused by vanishing gradients. These plain networks are trained with BN [16], which ensures forward propagated signals to have non-zero variances. We also verify that the backward propagated gradients exhibit healthy norms with BN. So neither forward nor backward signals vanish. In fact, the 34-layer plain net is still able to achieve competitive accuracy (Table 3), suggesting that the solver works to some extent. We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the

reducing of the training error³. The reason for such optimization difficulties will be studied in the future.

Residual Networks. Next we evaluate 18-layer and 34-layer residual nets (*ResNets*). The baseline architectures are the same as the above plain nets, expect that a shortcut connection is added to each pair of 3×3 filters as in Fig. 3 (right). In the first comparison (Table 2 and Fig. 4 right), we use identity mapping for all shortcuts and zero-padding for increasing dimensions (option A). So they have *no extra parameter* compared to the plain counterparts.

We have three major observations from Table 2 and Fig. 4. First, the situation is reversed with residual learning – the 34-layer ResNet is better than the 18-layer ResNet (by 2.8%). More importantly, the 34-layer ResNet exhibits considerably lower training error and is generalizable to the validation data. This indicates that the degradation problem is well addressed in this setting and we manage to obtain accuracy gains from increased depth.

Second, compared to its plain counterpart, the 34-layer

³We have experimented with more training iterations ($3 \times$) and still observed the degradation problem, suggesting that this problem cannot be feasibly addressed by simply using more iterations.

层名称 输出尺寸	18层	34层	50层	101层	152层
conv1	112×112	7×7, 64, 步长 2			
		3×3最大池化, 步长为2			
2次卷积	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
卷积3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	平均池化, 1000维全连接、Softmax			
浮点运算次数		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9

表1. ImageNet架构。括号内表示构建模块（详见图5），数字表示堆叠的块数。下采样通过步长为2的conv3_1、conv4_1和conv5_1卷积操作完成。

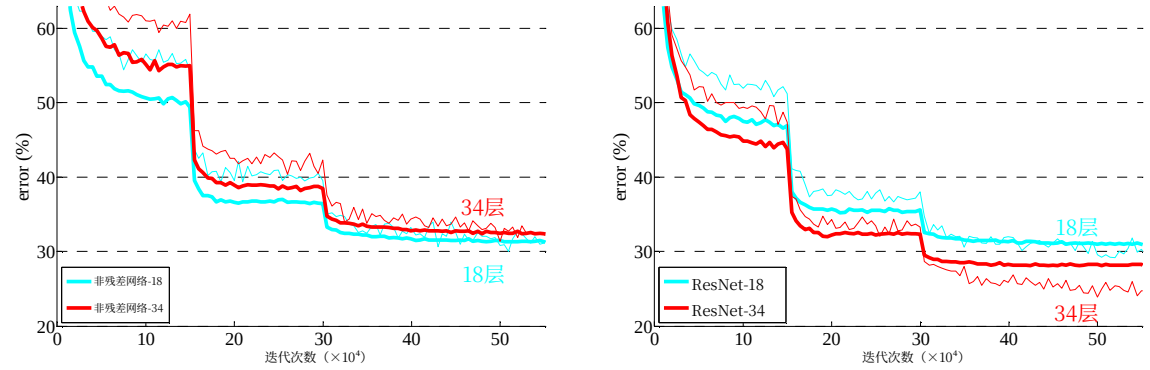


图4. 在**ImageNet**上的训练情况。细曲线表示训练误差，粗曲线表示中心裁剪的验证误差。左侧：18层与34层的普通网络。右侧：18层与34层的ResNet。此图中，相较于普通网络，残差网络未增加任何参数。

	普通网络	ResNet
18层	27.94	27.88
34层	28.54	25.03

表2. ImageNet验证集上的Top-1错误率（%，10-crop测试）。此处ResNets与其对应的普通网络相比无额外参数。图4展示了训练过程。

34层普通网络在整个训练过程中始终具有更高的训练误差，即使18层普通网络的解空间仅为34层网络解空间的子空间。

我们认为，这种优化困难是不太可能由梯度消失引起的。这些普通网络在使用 BN [16], 进行训练时，确保了前向传播的信号具有非零方差。我们还验证了在使用 BN 时，后向传播的梯度表现出稳定的范数。因此，前向与后向信号均未消失。事实上，34层的普通网络仍能达到具有竞争力的准确率（表3），表明求解器在一定程度上仍有效。我们推测，深层普通网络可能具有指数级缓慢的收敛速率，这影响了

训练误差的下降³。此类优化困难的成因将在未来进一步探讨。

残差网络。接下来我们评估18层和34-层残差网络 (*ResNets*)。基线架构与上述普通网络相同，不同之处在于参照图3右侧，为每对 3×3 滤波单元添加了捷径连接。在第一组对比实验（表2及图4右侧）中，我们对所有捷径均采用恒等映射，并对维度增加的情况使用零填充（选项A）。因此，与普通网络相比，它们具备无额外参数。

结合表2和图4，我们得出三项主要结论。首先，残差学习逆转了这一趋势——34层ResNet的表现优于18层ResNet（降低2.8%）。更重要的是，34层ResNet展现出大幅更低的训练误差，且能良好地泛化至验证数据。这说明在此设定下，退化问题已得到有效解决，我们通过增加网络深度成功获得了准确率的提升。

其次，与其对应的普通网络相比，该34层

³我们尝试了更多的训练迭代次数（ $3 \times$ ），但仍观察到退化现象，这表明该问题无法单纯通过增加迭代次数来有效解决。

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC’ 14)	7.32
GoogLeNet [44] (ILSVRC’ 14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

ResNet reduces the top-1 error by 3.5% (Table 2), resulting from the successfully reduced training error (Fig. 4 right vs. left). This comparison verifies the effectiveness of residual learning on extremely deep systems.

Last, we also note that the 18-layer plain/residual nets are comparably accurate (Table 2), but the 18-layer ResNet converges faster (Fig. 4 right vs. left). When the net is “not overly deep” (18 layers here), the current SGD solver is still able to find good solutions to the plain net. In this case, the ResNet eases the optimization by providing faster convergence at the early stage.

Identity vs. Projection Shortcuts. We have shown that

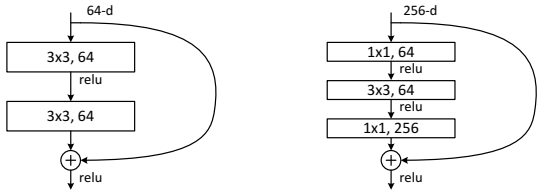


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

parameter-free, identity shortcuts help with training. Next we investigate projection shortcuts (Eqn.(2)). In Table 3 we compare three options: (A) zero-padding shortcuts are used for increasing dimensions, and all shortcuts are parameter-free (the same as Table 2 and Fig. 4 right); (B) projection shortcuts are used for increasing dimensions, and other shortcuts are identity; and (C) all shortcuts are projections.

Table 3 shows that all three options are considerably better than the plain counterpart. B is slightly better than A. We argue that this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and we attribute this to the extra parameters introduced by many (thirteen) projection shortcuts. But the small differences among A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem. So we do not use option C in the rest of this paper, to reduce memory/time complexity and model sizes. Identity shortcuts are particularly important for not increasing the complexity of the bottleneck architectures that are introduced below.

Deeper Bottleneck Architectures. Next we describe our deeper nets for ImageNet. Because of concerns on the training time that we can afford, we modify the building block as a *bottleneck* design⁴. For each residual function $\mathcal{F}$, we use a stack of 3 layers instead of 2 (Fig. 5). The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions. Fig. 5 shows an example, where both designs have similar time complexity.

The parameter-free identity shortcuts are particularly important for the bottleneck architectures. If the identity shortcut in Fig. 5 (right) is replaced with projection, one can show that the time complexity and model size are doubled, as the shortcut is connected to the two high-dimensional ends. So identity shortcuts lead to more efficient models for the bottleneck designs.

50-layer ResNet: We replace each 2-layer block in the

⁴Deeper *non*-bottleneck ResNets (*e.g.*, Fig. 5 left) also gain accuracy from increased depth (as shown on CIFAR-10), but are not as economical as the bottleneck ResNets. So the usage of bottleneck designs is mainly due to practical considerations. We further note that the degradation problem of plain nets is also witnessed for the bottleneck designs.

模型	top-1 错误率	top-5 错误率
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
普通网络-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

表3. ImageNet验证集上的错误率(%, **10-crop** 测试)。VGG-16基于我们的测试结果。ResNet-50/101/152均为方案B, 该方案仅在维度增加时使用投影。

方法	Top-1 错误率	Top-5 错误率
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

表 4. ImageNet 验证集上**单模型** 结果的错误率(%) (除[†] 在测试集上报告的结果外)。

方法	top-5 错误率 (测试)
VGG [41] (ILSVRC’ 14)	7.32
GoogLeNet [44] (ILSVRC’ 14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’ 15)	3.57

表5. **集成模型**的错误率(%). Top-5错误率基于ImageNet测试集, 由测试服务器报告。

ResNet 将 Top-1 错误率降低了 3.5% (表 2), 这得益于训练误差的成功降低 (图 4 右侧对比.左侧)。该对比验证了残差学习在极深系统中的有效性。

最后, 我们还注意到18层普通网络/残差网络的精度相当 (表2), 但18层ResNet收敛速度更快 (图4右侧与.左侧)。当网络 “不够深” (此处为18层) 时, 当前的SGD求解器仍能为普通网络找到良好的解。在此情况下, ResNet通过在初期提供更快收敛速度来降低优化难度。

恒等映射 与. 投影捷径. 我们已经表明

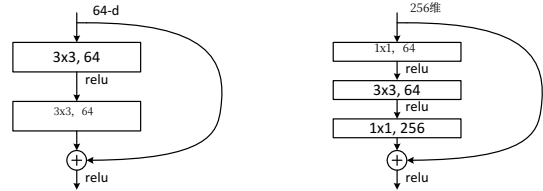


图 5. 用于 ImageNet 的更深残差函数 $\mathcal{F}$ 。左侧: 如 Fig. 3 所示、用于 ResNet-34 的基本模块 (在 56×56 特征图上)。右侧: 用于 ResNet-50/101/152 的 “瓶颈” 基本模块。

无参数的恒等快捷连接有助于训练。接下来我们研究投影快捷连接 (公式(2))。在表3中, 我们比较了三种方案: (A) 在维度增加时使用零填充快捷连接, 且所有快捷连接均为无参数形式 (与表2及图4右侧相同); (B) 在维度增加时使用投影快捷连接, 其余快捷连接采用恒等映射; 以及 (C) 所有快捷连接均为投影形式。

表 3 显示, 这三种方案均明显优于普通网络。B 略优于 A。我们认为这是因为 A 中补零的维度实际上并未参与残差学习。C 比 B 稍好, 我们将其归因于由多个 (共十三个) 投影捷径引入的额外参数。但 A/B/C 之间差异较小, 表明投影捷径对于解决退化问题并非必需。因此, 为降低内存/时间复杂度及模型规模, 本文后续不再使用选项 C。恒等捷径对于避免增加下文将介绍的瓶颈架构的复杂度尤为重要。

更深的瓶颈架构. 接下来我们介绍面向 ImageNet 的更深网络。考虑到可承受的训练时长, 我们将基本模块调整为瓶颈 设计⁴。对于每个残差函数 $\mathcal{F}$, 我们使用 3 层堆叠代替 2 层 (图 5)。这三层分别为 1×1 、 3×3 和 1×1 卷积, 其中 1×1 层负责降维随后升维 (恢复) 维度, 从而使 3×3 层成为输入/输出维度较小的瓶颈层。图 5 展示了其中一个实例, 两种设计的计算复杂度相当。

无参数的恒等捷径对于瓶颈架构尤为重要。如果将图5 (右) 中的恒等捷径替换为投影操作, 可证明其时间复杂度和模型规模将翻倍, 因为该捷径连接的是两个高维端。因此, 恒等捷径使瓶颈设计能产生更高效的模型。

50层ResNet: 我们将每个2层块替换为

⁴更深度的非-瓶颈ResNet (例如, 图5左) 随深度增加也能提升精度 (如在CIFAR-10上的结果所示), 但其效率不及瓶颈ResNet。因此, 采用瓶颈设计主要出于实际考量。我们还需指出, 普通网络的退化问题在瓶颈设计中同样存在。

34-layer net with this 3-layer bottleneck block, resulting in a 50-layer ResNet (Table 1). We use option B for increasing dimensions. This model has 3.8 billion FLOPs.

101-layer and 152-layer ResNets: We construct 101-layer and 152-layer ResNets by using more 3-layer blocks (Table 1). Remarkably, although the depth is significantly increased, the 152-layer ResNet (11.3 billion FLOPs) still has *lower complexity* than VGG-16/19 nets (15.3/19.6 billion FLOPs).

The 50/101/152-layer ResNets are more accurate than the 34-layer ones by considerable margins (Table 3 and 4). We do not observe the degradation problem and thus enjoy significant accuracy gains from considerably increased depth. The benefits of depth are witnessed for all evaluation metrics (Table 3 and 4).

Comparisons with State-of-the-art Methods. In Table 4 we compare with the previous best single-model results. Our baseline 34-layer ResNets have achieved very competitive accuracy. Our 152-layer ResNet has a single-model top-5 validation error of 4.49%. This single-model result outperforms all previous ensemble results (Table 5). We combine six models of different depth to form an ensemble (only with two 152-layer ones at the time of submitting). This leads to **3.57%** top-5 error on the test set (Table 5). *This entry won the 1st place in ILSVRC 2015.*

4.2. CIFAR-10 and Analysis

We conducted more studies on the CIFAR-10 dataset [20], which consists of 50k training images and 10k testing images in 10 classes. We present experiments trained on the training set and evaluated on the test set. Our focus is on the behaviors of extremely deep networks, but not on pushing the state-of-the-art results, so we intentionally use simple architectures as follows.

The plain/residual architectures follow the form in Fig. 3 (middle/right). The network inputs are 32×32 images, with the per-pixel mean subtracted. The first layer is 3×3 convolutions. Then we use a stack of $6n$ layers with 3×3 convolutions on the feature maps of sizes $\{32, 16, 8\}$ respectively, with $2n$ layers for each feature map size. The numbers of filters are $\{16, 32, 64\}$ respectively. The subsampling is performed by convolutions with a stride of 2. The network ends with a global average pooling, a 10-way fully-connected layer, and softmax. There are totally $6n+2$ stacked weighted layers. The following table summarizes the architecture:

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

When shortcut connections are used, they are connected to the pairs of 3×3 layers (totally $3n$ shortcuts). On this dataset we use identity shortcuts in all cases (*i.e.*, option A),

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean±std)” as in [43].

so our residual models have exactly the same depth, width, and number of parameters as the plain counterparts.

We use a weight decay of 0.0001 and momentum of 0.9, and adopt the weight initialization in [13] and BN [16] but with no dropout. These models are trained with a mini-batch size of 128 on two GPUs. We start with a learning rate of 0.1, divide it by 10 at 32k and 48k iterations, and terminate training at 64k iterations, which is determined on a 45k/5k train/val split. We follow the simple data augmentation in [24] for training: 4 pixels are padded on each side, and a 32×32 crop is randomly sampled from the padded image or its horizontal flip. For testing, we only evaluate the single view of the original 32×32 image.

We compare $n = \{3, 5, 7, 9\}$, leading to 20, 32, 44, and 56-layer networks. Fig. 6 (left) shows the behaviors of the plain nets. The deep plain nets suffer from increased depth, and exhibit higher training error when going deeper. This phenomenon is similar to that on ImageNet (Fig. 4, left) and on MNIST (see [42]), suggesting that such an optimization difficulty is a fundamental problem.

Fig. 6 (middle) shows the behaviors of ResNets. Also similar to the ImageNet cases (Fig. 4, right), our ResNets manage to overcome the optimization difficulty and demonstrate accuracy gains when the depth increases.

We further explore $n = 18$ that leads to a 110-layer ResNet. In this case, we find that the initial learning rate of 0.1 is slightly too large to start converging⁵. So we use 0.01 to warm up the training until the training error is below 80% (about 400 iterations), and then go back to 0.1 and continue training. The rest of the learning schedule is as done previously. This 110-layer network converges well (Fig. 6, middle). It has *fewer* parameters than other deep and thin

⁵With an initial learning rate of 0.1, it starts converging (<90% error) after several epochs, but still reaches similar accuracy.

34层网络结合此3层瓶颈块，从而构成50层ResNet（表1）。我们在维度增加时采用选项B。该模型的计算量为38亿FLOPs。

101层和152层ResNet：我们通过使用更多的3层块构建了101层和152层的ResNet（表1）。值得注意的是，尽管深度显著增加，152层ResNet（113亿FLOPs）的计算复杂度仍低于VGG-16/19网络（153/196亿FLOPs）。

50/101/152层ResNet的准确率相较于34层模型有显著提升（表3和表4）。我们未观察到退化现象，因此得益于网络深度的大幅增加，准确率获得了显著增益。所有评估指标均见证了深度带来的收益（表3和表4）。

与最先进方法的比较。在表4中，我们与之前的最佳单模型结果进行了对比。我们的基线34层ResNet已取得极具竞争力的准确率。我们的152层ResNet单模型Top-5验证错误率为4.49%。这一单模型结果优于所有此前的集成方法结果（表5）。我们将六个不同深度的模型组合成集成模型（提交时仅包含两个152层模型），从而在测试集上实现了**3.57%**的Top-5错误率（表5）。本方案荣获*ILSVRC 2015*冠军。

4.2. CIFAR-10 与分析

我们在 CIFAR-10 数据集[20], 上进行了进一步的研究，该数据集包含 10 个类别、共计 5 万张训练图像和 1 万张测试图像。我们展示了在训练集上训练、在测试集上评估的实验。我们的重点在于探究极深网络的行为特性，而非一味追求最先进（SOTA）的结果，因此我们刻意采用了如下简单的网络架构。

普通/残差架构遵循图 3（中/右）的形式。网络输入为 32×32 图像，并已减去像素均值。第一层为 3×3 卷积。随后，我们在尺寸分别为 $\{32, 16, 8\}$ 的特征图上使用由 $6n$ 层堆叠而成的模块，每个特征图尺寸对应 $2n$ 层，且包含 3×3 个卷积层。滤波器数量分别为 $\{16, 32, 64\}$ 。下采样通过步长为 2 的卷积操作实现。网络以全局平均池化、一个 10 分类的全连接层以及 Softmax 层结束。整个网络总共包含 $6n+2$ 层堆叠的权重层。下表对本架构进行了总结：

输出特征图尺寸	32×32	16×16	8×8
层数	$1+2n$	$2n$	$2n$
滤波器数量	16	32	64

当使用快捷连接时，它们连接至成对的 3×3 层（共 $3n$ 个捷径）。在该数据集上，我们在所有情况下均使用恒等捷径（即，选项A），

方法	错误率(%)		
Maxout [10]	9.38		
NIN [25]	8.81		
DSN [24]	8.22		
	层数	参数量	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
高速公路 [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

表6. **CIFAR-10** 测试集上的分类误差。所有方法均使用了数据增强。对于ResNet-110，我们运行了5次并展示“最佳（均值±标准差）”，格式同 [43]。

因此，我们的残差模型与对应的普通模型在深度、宽度及参数量上完全一致。

权重衰减为0.0001，动量为0.9，并采用 [13] 中的权重初始化方法与BN [16]，但不使用Dropout。这些模型在两块GPU上使用大小为128的mini-batch进行训练。初始学习率设为0.1，在32k和48k次迭代时按10倍衰减，于64k次迭代结束训练，此设置基于45k/5k的训练/验证集划分确定。训练过程遵循 [24] 中的简单数据增强策略：在图像四周各填充4个像素，随后从填充后的图像或其水平翻转图中随机抽取 32×32 尺寸的裁剪块。测试阶段，我们仅评估原始 32×32 图像的单一视图。

我们对比了 $n = \{3, 5, 7, 9\}$ ，由此得到20层、32层、44层和56层的网络。图6（左）展示了普通网络的表现。深层普通网络受限于增加的深度，且在网络加深时表现出更高的训练误差。这一现象与ImageNet（图4，左）和MNIST（见 [42]）上的情况类似，表明此类优化困难是一个根本性的问题。

图6（中）展示了ResNet的表现。与ImageNet的情况（图4，右）类似，我们的ResNet成功克服了优化困难，并随深度增加实现了准确率的提升。

我们进一步探索了 $n = 18$ ，由此构建了110层ResNet。在这种情况下，我们发现初始学习率为0.1时略大，难以启动收敛⁵。因此，我们使用0.01进行训练预热，直到训练误差低于80%（约400次迭代），然后恢复至0.1并继续训练。其余的学习计划与之前相同。该110层网络收敛良好（图6，中）。它具有更少的参数，比其他深而窄的

⁵采用0.1的初始学习率，模型在经过数个训练轮次后开始收敛（误差为<90%），但最终仍达到了相近的准确率。

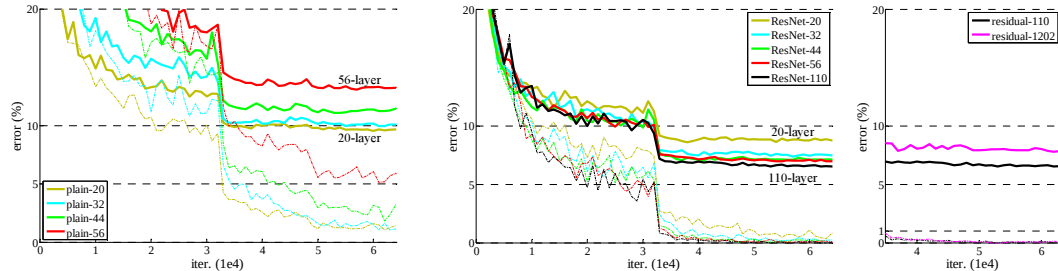


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

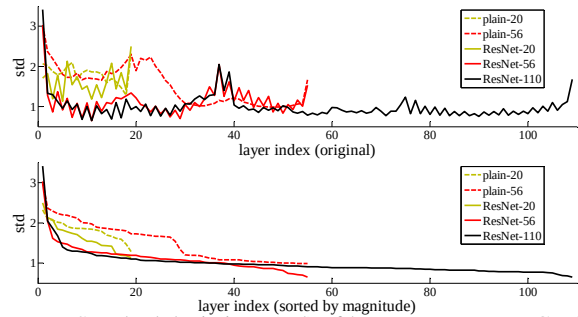


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

networks such as FitNet [35] and Highway [42] (Table 6), yet is among the state-of-the-art results (6.43%, Table 6).

Analysis of Layer Responses. Fig. 7 shows the standard deviations (std) of the layer responses. The responses are the outputs of each 3×3 layer, after BN and before other nonlinearity (ReLU/addition). For ResNets, this analysis reveals the response strength of the residual functions. Fig. 7 shows that ResNets have generally smaller responses than their plain counterparts. These results support our basic motivation (Sec.3.1) that the residual functions might be generally closer to zero than the non-residual functions. We also notice that the deeper ResNet has smaller magnitudes of responses, as evidenced by the comparisons among ResNet-20, 56, and 110 in Fig. 7. When there are more layers, an individual layer of ResNets tends to modify the signal less.

Exploring Over 1000 layers. We explore an aggressively deep model of over 1000 layers. We set $n = 200$ that leads to a 1202-layer network, which is trained as described above. Our method shows *no optimization difficulty*, and this 10^3 -layer network is able to achieve *training error* $< 0.1\%$ (Fig. 6, right). Its test error is still fairly good (7.93%, Table 6).

But there are still open problems on such aggressively deep models. The testing result of this 1202-layer network is worse than that of our 110-layer network, although both

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

have similar training error. We argue that this is because of overfitting. The 1202-layer network may be unnecessarily large (19.4M) for this small dataset. Strong regularization such as maxout [10] or dropout [14] is applied to obtain the best results ([10, 25, 24, 35]) on this dataset. In this paper, we use no maxout/dropout and just simply impose regularization via deep and thin architectures by design, without distracting from the focus on the difficulties of optimization. But combining with stronger regularization may improve results, which we will study in the future.

4.3. Object Detection on PASCAL and MS COCO

Our method has good generalization performance on other recognition tasks. Table 7 and 8 show the object detection baseline results on PASCAL VOC 2007 and 2012 [5] and COCO [26]. We adopt *Faster R-CNN* [32] as the detection method. Here we are interested in the improvements of replacing VGG-16 [41] with ResNet-101. The detection implementation (see appendix) of using both models is the same, so the gains can only be attributed to better networks. Most remarkably, on the challenging COCO dataset we obtain a 6.0% increase in COCO’s standard metric (mAP@[.5, .95]), which is a 28% relative improvement. This gain is solely due to the learned representations.

Based on deep residual nets, we won the 1st places in several tracks in ILSVRC & COCO 2015 competitions: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation. The details are in the appendix.

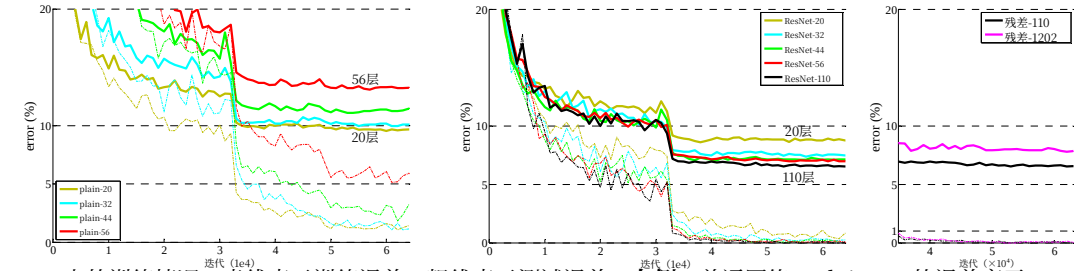


图6. 在**CIFAR-10**上的训练情况。虚线表示训练误差，粗线表示测试误差。**左侧**：普通网络。plain-110的误差高于60%，未予显示。**中间**：ResNets。**右侧**：具有110层和1202层的ResNets。

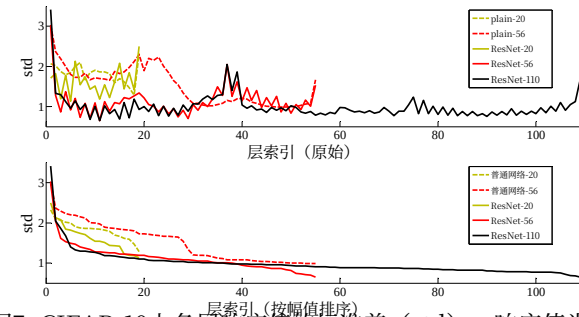


图7. CIFAR-10上各层响应值的标准差 (std)。响应值为每个 3×3 层经过BN处理、在进入非线性变换前的输出。**上部**：按各层的原始顺序排列。**下部**：响应值按降序排列。

训练数据	07+12	07++12
测试数据	VOC 07 测试	VOC 12 测试
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

表 7. 在 PASCAL VOC 2007/2012 测试集上使用 **基线** Faster R-CNN 的目标检测 mAP (%)。更好的结果请参见表 10 和 11。

指标	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

表 8. COCO 验证集上使用**基线** Faster R-CNN 的目标检测 mAP (%)。更优结果请参见表 9。

拥有相似的训练误差。我们认为这是由于过拟合导致的。针对此小规模数据集，该 1202 层网络可能显得过于庞大（19.4M）。为在该数据集上取得最佳结果 ([10, 25, 24, 35])，通常会应用诸如 maxout [10] 或 dropout [14] 等强正则化手段。本文不采用 maxout/dropout，仅通过精心设计深层且狭窄的架构来施加正则化，从而避免干扰对优化难点的研究重点。但结合更强的正则化或许能提升性能，这将是我们将来的研究方向。

4.3. PASCAL 与 MS COCO 上的目标检测

我们的方法在其他识别任务上展现出良好的泛化性能。表 7 和表 8 展示了在 PASCAL VOC 2007 和 2012 [5] 以及 COCO [26] 上的目标检测基线结果。我们采用 *Faster R-CNN* [32] 作为检测方法。此处我们关注将 VGG-16 [41] 替换为 ResNet-101 所带来的性能提升。两种模型的检测实现（见附录）完全相同，因此所获增益只能归因于更优的网络结构。尤为引人注目的是，在极具挑战性的 COCO 数据集上，我们在其标准指标（mAP@[.5,.95]）上提升了 6.0%，实现了 28% 的相对增长。这一提升完全得益于学习到的特征表示。

基于深度残差网络，我们在 ILSVRC 与 COCO 2015 竞赛的多个赛项中获得第一名：ImageNet 检测、ImageNet 定位、COCO 检测和 COCO 分割。详情请见附录。

网络（如 FitNet [35] 和 Highway [42]（表 6）），但仍属于最先进水平结果（6.43%，表 6）。

各层响应分析。图 7 展示了各层响应值的标准差 (std)。响应值是指每个 3×3 层在经过 BN 处理之后、于其他非线性操作（如 ReLU 或相加）之前的输出。针对 ResNets，该分析反映了残差函数的响应强度。由图 7 可知，ResNets 的响应值普遍小于对应的普通网络。这一结果支持了我们最初的基本动机（见第 3.1 节），即残差函数整体上可能比非残差函数更接近于零。此外我们还观察到，更深的 ResNet 表现出更小的响应幅度，这从图 7 中 ResNet-20、ResNet-56 与 ResNet-110 的对比中可得到验证。当网络层数增多时，ResNet 中的单个层对信号的改动通常会减少。

探索超过 1000 层。我们探索了一个极其深度的、拥有超过 1000 层的模型。我们设置 $n = 200$ 以构建一个 1202 层的网络，并按照上述方式进行训练。我们的方法展现出无优化困难，该 10^3 -层网络能够取得训练误差 $< 0.1\%$ （图 6 右侧）。其测试误差依然相当理想（7.93%，表 6）。

但对于这类极端深度的模型，仍存在许多悬而未决的问题。该 1202 层网络的测试结果不及我们的 110 层网络，尽管两者

References

- [1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] C. M. Bishop. *Neural networks for pattern recognition*. Oxford university press, 1995.
- [3] W. L. Briggs, S. F. McCormick, et al. *A Multigrid Tutorial*. Siam, 2000.
- [4] K. Chatfield, V. Lempitsky, A. Vedaldi, and A. Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In *BMVC*, 2011.
- [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, pages 303–338, 2010.
- [6] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware cnn model. In *ICCV*, 2015.
- [7] R. Girshick. Fast R-CNN. In *ICCV*, 2015.
- [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [9] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.
- [11] K. He and J. Sun. Convolutional neural networks at constrained time cost. In *CVPR*, 2015.
- [12] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *ECCV*, 2014.
- [13] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.
- [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012.
- [15] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [16] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [17] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *TPAMI*, 33, 2011.
- [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, and C. Schmid. Aggregating local image descriptors into compact codes. *TPAMI*, 2012.
- [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv:1408.5093*, 2014.
- [20] A. Krizhevsky. Learning multiple layers of features from tiny images. *Tech Report*, 2009.
- [21] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.
- [23] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.
- [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. *arXiv:1409.5185*, 2014.
- [25] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013.
- [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.
- [27] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

- [28] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. In *NIPS*, 2014.
- [29] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.
- [30] F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. In *CVPR*, 2007.
- [31] T. Raiko, H. Valpola, and Y. LeCun. Deep learning made easier by linear transformations in perceptrons. In *AISTATS*, 2012.
- [32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015.
- [33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. *arXiv:1504.06066*, 2015.
- [34] B. D. Ripley. *Pattern recognition and neural networks*. Cambridge university press, 1996.
- [35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.
- [36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *arXiv:1409.0575*, 2014.
- [37] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013.
- [38] N. N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. Technical report, 1998.
- [39] N. N. Schraudolph. Centering neural network gradient factors. In *Neural Networks: Tricks of the Trade*, pages 207–226. Springer, 1998.
- [40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.
- [41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [42] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015.
- [43] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. *1507.06228*, 2015.
- [44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [45] R. Szeliski. Fast surface interpolation using hierarchical basis functions. *TPAMI*, 1990.
- [46] R. Szeliski. Locally adapted hierarchical basis preconditioning. In *SIGGRAPH*, 2006.
- [47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. In *Neural Information Processing*, 2013.
- [48] A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.
- [49] W. Venables and B. Ripley. Modern applied statistics with s-plus. 1999.
- [50] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.

参考文献

- [1] Y. Bengio, P. Simard 和 P. Frasconi. 使用梯度下降难以学习长期依赖. IEEE Transactions on Neural Networks, 5(2):157–166, 1994. [2] C. M. Bishop. 用于模式识别的神经网络. Oxford University Press, 1995. [3] W. L. Briggs, S. F. McCormick 等. 多网格教程. Siam, 2000. [4] K. Chatfield, V. Lempitsky, A. Vedaldi 和 A. Zisserman. 细节决定成败：近期特征编码方法的评估. 发表于BMVC会议, 2011. [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn 和 A. Zisserman. Pascal视觉物体类别(VOC)挑战赛. IJCV, 页码303–338, 2010. [6] S. Gidaris 和 N. Komodakis. 通过多区域及语义分割感知CNN模型进行目标检测. 发表于ICCV会议, 2015. [7] R. Girshick. Fast R-CNN. 发表于ICCV会议, 2015. [8] R. Girshick, J. Donahue, T. Darrell 和 J. Malik. 用于精确目标检测与语义分割的丰富特征层次结构. 发表于CVPR会议, 2014. [9] X. Glorot 和 Y. Bengio. 理解训练深层前馈神经网络的困难. 发表于AISTATS会议, 2010. [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville 和 Y. Bengio. Maxout网络. arXiv:1302.4389, 2013. [11] K. He 和 J. Sun. 受限时耗下的卷积神经网络. 发表于CVPR会议, 2015. [12] K. He, X. Zhang, S. Ren 和 J. Sun. 视觉识别中深层卷积网络的空间金字塔池化. 发表于ECCV会议, 2014. [13] K. He, X. Zhang, S. Ren 和 J. Sun. 深入探究整流单元：在ImageNet分类上超越人类水平的性能. 发表于ICCV会议, 2015. [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever 和 R. R. Salakhutdinov. 避免特征检测器共适应以改进神经网络. arXiv:1207.0580, 2012. [15] S. Hochreiter 和 J. Schmidhuber. 长短期记忆. Neural computation, 9(8):1735–1780, 1997. [16] S. Ioffe 和 C. Szegedy. 批归一化：通过减少内部协变量偏移加速深层网络训练. 发表于ICML会议, 2015. [17] H. Jegou, M. Douze 和 C. Schmid. 用于最近邻搜索的乘积量化. TPAMI, 33, 2011. [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez 和 C. Schmid. 将局部图像描述符聚合为紧凑代码. TPAMI, 2012. [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama 和 T. Darrell. Caffe：用于快速特征嵌入的卷积架构. arXiv:1408.5093, 2014. [20] A. Krizhevsky. 从微小图像中学习多层特征. 技术报告, 2009. [21] A. Krizhevsky, I. Sutskever 和 G. Hinton. 使用深层卷积神经网络进行ImageNet分类. 发表于NIPS会议, 2012. [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard 和 L. D. Jackel. 反向传播应用于手写邮政编码识别. Neural computation, 1989. [23] Y. LeCun, L. Bottou, G. B. Orr 和 K.-R. Müller. 高效反向传播. 收录于《Neural Networks: Tricks of the Trade》, 页码9–50. Springer, 1998. [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang 和 Z. Tu. 深度监督网络. arXiv:1409.5185, 2014. [25] M. Lin, Q. Chen 和 S. Yan. 网中网. arXiv:1312.4400, 2013. [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár 和 C. L. Zitnick. 微软COCO：上下文中的通用物体. 发表于ECCV会议. 2014. [27] J. Long, E. Shelhamer 和 T. Darrell. 用于语义分割的全卷积网络. 发表于CVPR会议, 2015.

- [28] G. Mont´ufar, R. Pascanu, K. Cho, and Y. Bengio. 深度神经网络的线性区域数量. 发表于 NIPS, 2014.[29] V. Nair and G. E. Hinton. 整流线性单元改善受限玻尔兹曼机. 发表于ICML, 2010.[30] F. Perronnin and C. Dance. 用于图像分类的视觉词袋上的 Fisher 核. 发表于 CVPR, 2007.[31] T. Raiko, H. Valpola, and Y. LeCun. 通过感知机的线性变换使深度学习更简便. 发表于 AISTATS, 2012.[32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN：利用区域提议网络迈向实时目标检测. 发表于 NIPS, 2015.[33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. 基于卷积特征图的目标检测网络. arXiv:1504.06066, 2015. [34] B. D. Ripley. 模式识别与神经网络. 剑桥大学出版社, 1996. [35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets：为薄深层网络提供的提示. 发表于 ICLR, 2015.[36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. ImageNet 大规模视觉识别挑战. arXiv:1409.0575, 2014.[37] A. M. Saxe, J. L. McClelland, and S. Ganguli. 深度线性神经网络中学习非线性动力学的精确解. arXiv:1312.6120, 2013.[38] N. N. Schraudolph. 基于因子中心化分解的加速梯度下降法. 技术报告, 1998.[39] N. N. Schraudolph. 神经网络梯度因子的中心化. 收录于《Neural Networks: Tricks of the Trade》, 第 207–226 页. Springer, 1998.[40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. Le-Cun. Overfeat：基于卷积网络的统一识别、定位与检测. 发表于ICLR, 2014.[41] K. Simonyan and A. Zisserman. 面向大规模图像识别的极深卷积网络. 发表于ICLR, 2015.[42] R. K. Srivastava, K. Greff, and J. Schmidhuber. 高速公路网络. arXiv:1505.00387, 2015.[43] R. K. Srivastava, K. Greff, and J. Schmidhuber. 训练极深网络. 1507.06228, 2015. [44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. 借助卷积加深网络. 发表于 CVPR, 2015.[45] R. Szeliski. 使用分层基函数的快速表面插值. TPAMI, 1990.[46] R. Szeliski. 局部自适应分层基预处理. 发表于 SIGGRAPH, 2006.[47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. 推动随机梯度向二阶方法演进——非线性变换下的反向传播学习. 收录于 Neural Information Processing, 2013.[48] A. Vedaldi and B. Fulkerson. VLFeat：一款开源且跨平台的计算机视觉算法库, 2008.[49] W. Venables and B. Ripley. 基于 S-Plus 的现代应用统计学. 1999.[50] M. D. Zeiler and R. Fergus. 可视化与理解卷积神经网络. 发表于 ECCV, 2014.

A. Object Detection Baselines

In this section we introduce our detection method based on the baseline Faster R-CNN [32] system. The models are initialized by the ImageNet classification models, and then fine-tuned on the object detection data. We have experimented with ResNet-50/101 at the time of the ILSVRC & COCO 2015 detection competitions.

Unlike VGG-16 used in [32], our ResNet has no hidden fc layers. We adopt the idea of “Networks on Conv feature maps” (NoC) [33] to address this issue. We compute the full-image shared conv feature maps using those layers whose strides on the image are no greater than 16 pixels (*i.e.*, conv1, conv2_x, conv3_x, and conv4_x, totally 91 conv layers in ResNet-101; Table 1). We consider these layers as analogous to the 13 conv layers in VGG-16, and by doing so, both ResNet and VGG-16 have conv feature maps of the same total stride (16 pixels). These layers are shared by a region proposal network (RPN, generating 300 proposals) [32] and a Fast R-CNN detection network [7]. RoI pooling [7] is performed before conv5_1. On this RoI-pooled feature, all layers of conv5_x and up are adopted for each region, playing the roles of VGG-16’s fc layers. The final classification layer is replaced by two sibling layers (classification and box regression [7]).

For the usage of BN layers, after pre-training, we compute the BN statistics (means and variances) for each layer on the ImageNet training set. Then the BN layers are fixed during fine-tuning for object detection. As such, the BN layers become linear activations with constant offsets and scales, and BN statistics are not updated by fine-tuning. We fix the BN layers mainly for reducing memory consumption in Faster R-CNN training.

PASCAL VOC

Following [7, 32], for the PASCAL VOC 2007 *test* set, we use the 5k *trainval* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07+12”). For the PASCAL VOC 2012 *test* set, we use the 10k *trainval+test* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07++12”). The hyper-parameters for training Faster R-CNN are the same as in [32]. Table 7 shows the results. ResNet-101 improves the mAP by >3% over VGG-16. This gain is solely because of the improved features learned by ResNet.

MS COCO

The MS COCO dataset [26] involves 80 object categories. We evaluate the PASCAL VOC metric (mAP @ IoU = 0.5) and the standard COCO metric (mAP @ IoU = .5:.05:.95). We use the 80k images on the train set for training and the 40k images on the val set for evaluation. Our detection system for COCO is similar to that for PASCAL VOC. We train the COCO models with an 8-GPU implementation, and thus the RPN step has a mini-batch size of

8 images (*i.e.*, 1 per GPU) and the Fast R-CNN step has a mini-batch size of 16 images. The RPN step and Fast R-CNN step are both trained for 240k iterations with a learning rate of 0.001 and then for 80k iterations with 0.0001.

Table 8 shows the results on the MS COCO validation set. ResNet-101 has a 6% increase of mAP@[.5, .95] over VGG-16, which is a 28% relative improvement, solely contributed by the features learned by the better network. Remarkably, the mAP@[.5, .95]’s absolute increase (6.0%) is nearly as big as mAP@.5’s (6.9%). This suggests that a deeper network can improve both recognition and localization.

B. Object Detection Improvements

For completeness, we report the improvements made for the competitions. These improvements are based on deep features and thus should benefit from residual learning.

MS COCO

Box refinement. Our box refinement partially follows the iterative localization in [6]. In Faster R-CNN, the final output is a regressed box that is different from its proposal box. So for inference, we pool a new feature from the regressed box and obtain a new classification score and a new regressed box. We combine these 300 new predictions with the original 300 predictions. Non-maximum suppression (NMS) is applied on the union set of predicted boxes using an IoU threshold of 0.3 [8], followed by box voting [6]. Box refinement improves mAP by about 2 points (Table 9).

Global context. We combine global context in the Fast R-CNN step. Given the full-image conv feature map, we pool a feature by global Spatial Pyramid Pooling [12] (with a “single-level” pyramid) which can be implemented as “RoI” pooling using the entire image’s bounding box as the RoI. This pooled feature is fed into the post-RoI layers to obtain a global context feature. This global feature is concatenated with the original per-region feature, followed by the sibling classification and box regression layers. This new structure is trained end-to-end. Global context improves mAP@.5 by about 1 point (Table 9).

Multi-scale testing. In the above, all results are obtained by single-scale training/testing as in [32], where the image’s shorter side is $s = 600$ pixels. Multi-scale training/testing has been developed in [12, 7] by selecting a scale from a feature pyramid, and in [33] by using maxout layers. In our current implementation, we have performed multi-scale *testing* following [33]; we have not performed multi-scale training because of limited time. In addition, we have performed multi-scale testing only for the Fast R-CNN step (but not yet for the RPN step). With a trained model, we compute conv feature maps on an image pyramid, where the image’s shorter sides are $s \in \{200, 400, 600, 800, 1000\}$.

A. 目标检测基线

本节我们将介绍基于基准 Faster R-CNN [32]系统的检测方法。模型首先由 ImageNet 分类模型初始化，随后在目标检测数据上进行微调。在 ILSVRC 与 COCO 2015 目标检测竞赛期间，我们针对 ResNet-50/101 开展了实验。

与在 [32], 中使用的VGG-16不同，我们的ResNet没有隐藏的全连接（fc）层。为解决这一问题，我们采用了“卷积特征图上的网络”（NoC）思想 [33]。我们利用图像步长不超过16像素的层来计算全图共享的卷积特征图（即：*i.e.*, conv1, conv2 x, conv3 x和conv4 x, ResNet-101中总计包含91个卷积层；表1）。我们将这些层视为与VGG-16中的13个卷积层等效，如此一来，ResNet与VGG-16的卷积特征图便拥有相同的总体步长（16像素）。这些层由区域提议网络（RPN，负责生成300个候选区域）[32] 和Fast R-CNN检测网络 [7]共享。在conv5_1之前进行RoI池化 [7]操作。在此RoI池化后的特征上，针对每个区域都会调用所有conv5 x及更高层级的网络，以发挥VGG-16中全连接层的作用。最终的分类层被替换为两个并列的网络分支（分类与边界框回归 [7]）。

关于BN层的使用，在预训练之后，我们在 ImageNet训练集上计算每一层的BN统计量（均值和方差）。然后在目标检测的微调过程中固定这些BN层。因此，BN层变成了带有固定偏移量和缩放因子的线性激活，且其统计量不会在微调过程中更新。我们固定BN层主要是为了降低Faster R-CNN训练过程中的显存消耗。

PASCAL VOC

依据 [7, 32], , 针对PASCAL VOC 2007 测试 集，我们使用VOC 2007中的5k张训练验证 图像以及VOC 2012中的16k张训练验证 图像进行训练（“07+12”）。对于PASCAL VOC 2012 测试 集，我们使用VOC 2007中的10k张训练验证+测试图像以及VOC 2012中的16k张训练验证 图像进行训练（“07++12”）。Faster R-CNN的训练超参数与 [32]中保持一致。表7展示了结果。与VGG-16相比，ResNet-101的mAP提升了 >3%。这一提升完全归因于ResNet学习到的更优特征。

MS COCO

MS COCO数据集 [26] 包含80个物体类别。我们评估PASCAL VOC指标（mAP@IoU = 0.5）和标准 COCO指标（mAP@IoU =.5:.05:.95）。我们使用训练集上的8万张图片进行训练，并使用验证集上的4万张图片进行评估。我们在COCO上的检测系统与在PASCAL VOC上的类似。我们基于8块GPU的实现来训练COCO模型，因此RPN步骤的mini-batch大小为

8张图片（即每块GPU对应1张），且Fast R-CNN步骤的 mini-batch大小为16张图片。RPN步骤和Fast R-CNN步骤均以 0.001的学习率训练24万次迭代，随后再以0.0001的学习率训练8万次迭代。

表8展示了在MS COCO验证集上的结果。与VGG-16相比，ResNet-101在mAP@[.5, .95]指标上提升了6%，实现了28%的相对提升，这完全归功于更优网络所学习的特征。值得注意的是，mAP@[.5, .95]的绝对提升幅度（6.0%）几乎与mAP@0.5的提升幅度（6.9%）相当。这表明更深层次的网络能够同时改善检测识别与定位精度。

B. 目标检测改进

为求完整，我们报告了针对该竞赛所做的改进。这些改进均基于深度特征，因此应能从残差学习中受益。

MS COCO

*边界框精修。*我们的边界框精修部分借鉴了 [6]中的迭代定位方法。在Faster R-CNN中，最终输出是回归修正后的边界框，与原始的建议框有所不同。因此，在推理时，我们从回归后的边界框中提取新的特征，进而得到新的分类得分和新的回归边界框。我们将这300个新预测结果与原本的300个预测结果合并。对预测边界框的并集执行非极大值抑制（NMS），IoU阈值设为0.3 [8], , 随后进行边界框投票 [6]。边界框精修使mAP提升了约2个百分点（见表9）。

*全局上下文。*我们在Fast R-CNN阶段融合全局上下文。给定整幅图像的卷积特征图，我们通过全局空间金字塔池化 [12]（具有“单层”金字塔）对特征进行池化，该操作可通过将整图边界框用作RoI来实现 “RoI” 池化。该池化后的特征被送入RoI后续层，以获取全局上下文特征。该全局特征与原始的区域特征拼接，随后进入同级分类与边界框回归层。该新结构采用端到端方式训练。全局上下文的加入使 mAP@.5提升了约1分（表9）。

*多尺度测试。*上述结果均是通过单尺度训练/测试得到的，如 [32], 中所述，其中图像短边为 $s = 600$ 像素。多尺度训练/测试已在 [12, 7]中通过从特征金字塔中选择一个尺度实现，并在 [33] 中使用maxout层实现。在当前实现中，我们参照 [33]进行了多尺度测试；由于时间有限，我们尚未进行多尺度训练。此外，我们仅针对Fast R-CNN阶段进行了多尺度测试（尚未在RPN阶段进行测试）。使用训练好的模型时，我们将在图像金字塔上计算卷积特征图，其中图像的短边设置为 $s \in \{200, 400, 600, 800, 1000\}$ 。

training data	COCO train		COCO trainval	
test data	COCO val		COCO test-dev	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
baseline Faster R-CNN (VGG-16)	41.5	21.2		
baseline Faster R-CNN (ResNet-101)	48.4	27.2		
+box refinement	49.9	29.9		
+context	51.1	30.0	53.3	32.2
+multi-scale testing	53.8	32.5	55.7	34.9
ensemble			59.0	37.4

Table 9. Object detection improvements on MS COCO using Faster R-CNN and ResNet-101.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
baseline	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
baseline+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

Table 10. Detection results on the PASCAL VOC 2007 test set. The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
baseline	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
baseline+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

Table 11. Detection results on the PASCAL VOC 2012 test set (<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>). The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

We select two adjacent scales from the pyramid following [33]. RoI pooling and subsequent layers are performed on the feature maps of these two scales [33], which are merged by maxout as in [33]. Multi-scale testing improves the mAP by over 2 points (Table 9).

Using validation data. Next we use the 80k+40k trainval set for training and the 20k test-dev set for evaluation. The test-dev set has no publicly available ground truth and the result is reported by the evaluation server. Under this setting, the results are an mAP@.5 of 55.7% and an mAP@[.5, .95] of 34.9% (Table 9). This is our single-model result.

Ensemble. In Faster R-CNN, the system is designed to learn region proposals and also object classifiers, so an ensemble can be used to boost both tasks. We use an ensemble for proposing regions, and the union set of proposals are processed by an ensemble of per-region classifiers. Table 9 shows our result based on an ensemble of 3 networks. The mAP is 59.0% and 37.4% on the test-dev set. *This result won the 1st place in the detection task in COCO 2015.*

PASCAL VOC

We revisit the PASCAL VOC dataset based on the above model. With the single model on the COCO dataset (55.7% mAP@.5 in Table 9), we fine-tune this model on the PASCAL VOC sets. The improvements of box refinement, context, and multi-scale testing are also adopted. By doing so

	val2	test
GoogLeNet [44] (ILSVRC’ 14)	-	43.9
our single model (ILSVRC’ 15)	60.5	58.8
our ensemble (ILSVRC’ 15)	63.6	62.1

Table 12. Our results (mAP, %) on the ImageNet detection dataset. Our detection system is Faster R-CNN [32] with the improvements in Table 9, using ResNet-101.

we achieve 85.6% mAP on PASCAL VOC 2007 (Table 10) and 83.8% on PASCAL VOC 2012 (Table 11)⁶. The result on PASCAL VOC 2012 is 10 points higher than the previous state-of-the-art result [6].

ImageNet Detection

The ImageNet Detection (DET) task involves 200 object categories. The accuracy is evaluated by mAP@.5. Our object detection algorithm for ImageNet DET is the same as that for MS COCO in Table 9. The networks are pre-trained on the 1000-class ImageNet classification set, and are fine-tuned on the DET data. We split the validation set into two parts (val1/val2) following [8]. We fine-tune the detection models using the DET training set and the val1 set. The val2 set is used for validation. We do not use other ILSVRC 2015 data. Our single model with ResNet-101 has

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>, submitted on 2015-11-26.

训练数据	COCO训练集		COCO训练验证集	
测试数据	COCO验证集		COCO test-dev集	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
基线 Faster R-CNN (VGG-16)	41.5	21.2		
基线 Faster R-CNN (ResNet-101)	48.4	27.2		
+边界框细化	49.9	29.9		
+上下文	51.1	30.0	53.3	32.2
+多尺度测试	53.8	32.5	55.7	34.9
集成			59.0	37.4

表 9. 使用 Faster R-CNN 和 ResNet-101 在 MS COCO 数据集上实现的目标检测改进。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	摩托车	人	盆栽	绵羊	sofa	火车	tv
基线	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
基线	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
基线+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

表10。PASCAL VOC 2007测试集上的检测结果。基线为Faster R-CNN系统。“基线+++”系统包含边界框精调、上下文信息及多尺度测试（见表9）。

系统	net	data	mAP	areo	bike	bird	boat	瓶	bus	car	cat	椅	cow	桌	dog	马	摩托车	人	植物	羊	sofa	火车	tv
基线	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
基线	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
基线+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

表 11. 在 PASCAL VOC 2012 测试集上的检测结果 (<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>)。基线为 Faster R-CNN 系统。“基线+++”系统包含表 9 中的框精修、上下文信息及多尺度测试。

我们参照[33]从金字塔中选取两个相邻的尺度。在这两个尺度对应的特征图（[33].）上执行RoI池化及后续层的操作，并通过maxout将它们融合，如[33]所示。多尺度测试使mAP提升了超过2个百分点（见表9）。

使用验证数据。接下来，我们使用 80k+40k 的 trainval 集进行训练，并采用 20k 的 test-dev 集进行评估。test-dev 集无公开的真实标注数据，结果由评估服务器报告。在此设置下，结果为 mAP@.5 达 55.7%，且 mAP@[.5, .95] 为 34.9%（表 9）。这是我们的单模型结果。

集成。在 Faster R-CNN 中，系统被设计用于学习区域提议和目标分类器，因此可采用集成策略来同时提升这两项任务。我们使用集成模型进行区域提议，并将提议区域的并集交由一组每区域分类器的集成模型进行处理。表 9展示了基于 3 个网络集成的结果。在test-dev 集上，mAP 分别为 59.0% 和 37.4%。该结果在2015 年 COCO 检测任务中荣获第 1 名。

PASCAL VOC

基于上述模型，我们重新在PASCAL VOC数据集上进行了实验。采用在COCO数据集上的单一模型（表9显示mAP@.5为55.7%），我们在PASCAL VOC数据集上对该模型进行微调。同时，我们也应用了边界框优化、上下文信息及多尺度测试等改进方法。借助上述操作

	val2	test
GoogLeNet [44] (ILSVRC’ 14)	-	43.9
我们的单一模型（ILSVRC’ 15）	60.5	58.8
我们的集成模型（ILSVRC’ 15）	63.6	62.1

表12。我们在ImageNet检测数据集上的结果（mAP，%）。我们的检测系统基于Faster R-CNN [32]，结合表9中的改进，并使用 ResNet-101。

我们在PASCAL VOC 2007上取得了85.6%的mAP（表10），在PASCAL VOC 2012上达到了83.8%（表11）⁶。该结果在PASCAL VOC 2012上较先前的最先进水平 [6]提升了10个百分点。

ImageNet 检测

ImageNet 检测（DET）任务涵盖 200 个物体类别。准确率通过 mAP@.5 进行评估。我们针对 ImageNet DET 的目标检测算法与表 9 中 MS COCO 的算法相同。网络在 1000 类 ImageNet 分类集上预训练，并在 DET 数据上进行微调。我们参照 [8] 将验证集分为两部分 (val1/val2)。我们利用 DET 训练集和 val1集对检测模型进行微调。val2 集用于验证。不使用其他 ILSVRC 2015 数据。采用 ResNet-101 的单模型具有

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>，于2015-11-26提交。

LOC method	LOC network	testing	LOC error on GT CLS	classification network	top-5 LOC error on predicted CLS
VGG’s [41]	VGG-16	1-crop	33.1 [41]		
RPN	ResNet-101	1-crop	13.3		
RPN	ResNet-101	dense	11.7		
RPN	ResNet-101	dense		ResNet-101	14.4
RPN+RCNN	ResNet-101	dense		ResNet-101	10.6
RPN+RCNN	ensemble	dense		ensemble	8.9

Table 13. Localization error (%) on the ImageNet validation. In the column of “LOC error on GT class” ([41]), the ground truth class is used. In the “testing” column, “1-crop” denotes testing on a center crop of 224×224 pixels, “dense” denotes dense (fully convolutional) and multi-scale testing.

58.8% mAP and our ensemble of 3 models has 62.1% mAP on the DET test set (Table 12). *This result won the 1st place in the ImageNet detection task in ILSVRC 2015*, surpassing the second place by **8.5 points** (absolute).

C. ImageNet Localization

The ImageNet Localization (LOC) task [36] requires to classify and localize the objects. Following [40, 41], we assume that the image-level classifiers are first adopted for predicting the class labels of an image, and the localization algorithm only accounts for predicting bounding boxes based on the predicted classes. We adopt the “per-class regression” (PCR) strategy [40, 41], learning a bounding box regressor for each class. We pre-train the networks for ImageNet classification and then fine-tune them for localization. We train networks on the provided 1000-class ImageNet training set.

Our localization algorithm is based on the RPN framework of [32] with a few modifications. Unlike the way in [32] that is category-agnostic, our RPN for localization is designed in a *per-class* form. This RPN ends with two sibling 1×1 convolutional layers for binary classification (*cls*) and box regression (*reg*), as in [32]. The *cls* and *reg* layers are both in a *per-class* from, in contrast to [32]. Specifically, the *cls* layer has a 1000-d output, and each dimension is *binary logistic regression* for predicting being or not being an object class; the *reg* layer has a 1000×4-d output consisting of box regressors for 1000 classes. As in [32], our bounding box regression is with reference to multiple translation-invariant “anchor” boxes at each position.

As in our ImageNet classification training (Sec. 3.4), we randomly sample 224×224 crops for data augmentation. We use a mini-batch size of 256 images for fine-tuning. To avoid negative samples being dominate, 8 anchors are randomly sampled for each image, where the sampled positive and negative anchors have a ratio of 1:1 [32]. For testing, the network is applied on the image fully-convolutionally.

Table 13 compares the localization results. Following [41], we first perform “oracle” testing using the ground truth class as the classification prediction. VGG’s paper [41] re-

method	top-5 localization err	
	val	test
OverFeat [40] (ILSVRC’ 13)	30.0	29.9
GoogLeNet [44] (ILSVRC’ 14)	-	26.7
VGG [41] (ILSVRC’ 14)	26.9	25.3
ours (ILSVRC’ 15)	8.9	9.0

Table 14. Comparisons of localization error (%) on the ImageNet dataset with state-of-the-art methods.

ports a center-crop error of 33.1% (Table 13) using ground truth classes. Under the same setting, our RPN method using ResNet-101 net significantly reduces the center-crop error to 13.3%. This comparison demonstrates the excellent performance of our framework. With dense (fully convolutional) and multi-scale testing, our ResNet-101 has an error of 11.7% using ground truth classes. Using ResNet-101 for predicting classes (4.6% top-5 classification error, Table 4), the top-5 localization error is 14.4%.

The above results are only based on the *proposal network* (RPN) in Faster R-CNN [32]. One may use the *detection network* (Fast R-CNN [7]) in Faster R-CNN to improve the results. But we notice that on this dataset, one image usually contains a single dominate object, and the proposal regions highly overlap with each other and thus have very similar RoI-pooled features. As a result, the image-centric training of Fast R-CNN [7] generates samples of small variations, which may not be desired for stochastic training. Motivated by this, in our current experiment we use the original R-CNN [8] that is RoI-centric, in place of Fast R-CNN.

Our R-CNN implementation is as follows. We apply the per-class RPN trained as above on the training images to predict bounding boxes for the ground truth class. These predicted boxes play a role of class-dependent proposals. For each training image, the highest scored 200 proposals are extracted as training samples to train an R-CNN classifier. The image region is cropped from a proposal, warped to 224×224 pixels, and fed into the classification network as in R-CNN [8]. The outputs of this network consist of two sibling fc layers for *cls* and *reg*, also in a per-class form. This R-CNN network is fine-tuned on the training set using a mini-batch size of 256 in the RoI-centric fashion. For testing, the RPN generates the highest scored 200 proposals for each predicted class, and the R-CNN network is used to update these proposals’ scores and box positions.

This method reduces the top-5 localization error to 10.6% (Table 13). This is our single-model result on the validation set. Using an ensemble of networks for both classification and localization, we achieve a top-5 localization error of 9.0% on the test set. This number significantly outperforms the ILSVRC 14 results (Table 14), showing a 64% relative reduction of error. *This result won the 1st place in the ImageNet localization task in ILSVRC 2015*.

LOC 方法	LOC 网络	测试	定位误差在真实分类上	分类网络	top-5 定位误差在预测类别上
VGG 的 [41]	VGG-16	单裁剪	33.1 [41]		
RPN	ResNet-101	单裁剪	13.3		
RPN	ResNet-101	密集	11.7		
RPN	ResNet-101	密集		ResNet-101	14.4
RPN+RCNN	ResNet-101	密集		ResNet-101	10.6
RPN+RCNN	集成	稠密		集成	8.9

表13. ImageNet验证集上的定位误差（%）。在“GT类定位误差”（[41]）列中，使用的是真实类别。在“测试”列中, “1-crop”表示对大小为 224×224 像素的中心裁剪进行测试, “dense”表示采用密集（全卷积）及多尺度测试。

在 DET 测试集（表 12）上取得了 58.8% 的 mAP，由 3 个模型组成的集成模型达到了 62.1% 的 mAP该结果在 ILSVRC 2015 的 ImageNet 检测任务中获得第 1 名，领先第二名 **8.5 分**（绝对数值）。

C. ImageNet定位

ImageNet 定位（LOC）任务 [36] 要求对物体进行分类与定位。遵循 [40, 41], 的设定，我们假设首先采用图像级分类器来预测图像的分类标签，而定位算法仅负责根据预测出的类别来预测边界框。我们采用“逐类回归”（PCR）策略 [40, 41], 为每个类别学习一个边界框回归器。我们首先为 ImageNet 分类任务预训练网络，然后针对定位任务对其进行微调。我们在提供的含 1000 个类别的 ImageNet 训练集上训练网络。

我们的定位算法基于 [32] 的 RPN 框架，并进行了少量修改。与 [32] 中无关类别的检测方式不同，我们用于定位的 RPN 设计为 逐类 形式。该 RPN 末尾包含两个并列的 1×1 卷积层，分别用于二分类（分类）和框回归（回归），具体实现如 [32]所示。与 [32]相反，分类和回归 层均采用 逐类 形式。具体而言，分类层的输出维度为 1000，每个维度使用 二元逻辑回归 来预测是否属于某个物体类别；回归 层的输出维度为 1000×4-d，由涵盖 1000 个类别的框回归器组成。如同 [32], 所述，我们的边界框回归在每个位置参考了多个平移不变的“锚框”。

与我们的ImageNet分类训练（第3.4节）一致，我们随机采样 224×224 个裁剪区域用于数据增强。微调时采用大小为256的Mini-batch。为避免负样本占主导，每张图像随机采样8个锚框，其中正负采样锚框比例为1: 1 [32]。测试时，网络以全卷积方式作用于整张图像。

表13对比了定位结果。根据[41],，我们首先以真实类别作为分类预测来进行“Oracle”测试。VGG的论文 [41] 报

方法	Top-5定位误差	
	val	test
OverFeat [40] (ILSVRC’ 13)	30.0	29.9
GoogLeNet [44] (ILSVRC’ 14)	-	26.7
VGG [41](ILSVRC’ 14)	26.9	25.3
我们的方法 (ILSVRC’ 15)	8.9	9.0

表14. 与最先进方法在ImageNet数据集上的定位误差（%）对比。

告中心裁剪错误率为33.1%（表13），且使用了真实类别。在相同设置下，我们采用ResNet-101网络的RPN方法将中心裁剪错误率显著降至13.3%。此对比彰显了本框架的卓越性能。在进行密集（全卷积）与多尺度测试时，我们的ResNet-101在真实类别上的错误率为11.7%。利用 ResNet-101进行类别预测时（Top-5分类错误率为4.6%，见表4），其Top-5定位错误率为14.4%。

上述结果仅基于Faster R-CNN [32]中的区域建议网络(RPN)。可以使用Faster R-CNN中的检测网络（Fast R-CNN [7]）提升结果。但我们注意到，该数据集的每张图像通常仅包含一个主导物体，区域建议框之间高度重叠，因此其RoI池化特征极为相似。结果是，Fast R-CNN [7]的以图像为中心的训练所生成的样本变化幅度很小，这可能并非随机训练所需。受此启发，在当前实验中，我们使用以RoI为中心的原始R-CNN [8]来替代Fast R-CNN。

我们的R-CNN实现流程如下。我们将上述训练好的逐类RPN应用于训练图像，以预测真实类别的边界框。这些预测框充当类别相关提案。对于每张训练图像，提取得分最高的200个提案作为训练样本，用于训练R-CNN分-类器。从提案中裁剪出图像区域，将其变换至 224×224像素尺寸后，按R-CNN [8]的方式输入分类网络。该网络的输出包含两个并列的全连接层，分别对应分类和回归，同样采用逐类形式。该 R-CNN网络以RoI为中心的方式在训练集上进行微调，mini-batch大小为256。测试阶段，RPN为每个预测类别生成得分最高的200个提案，随后利用R-CNN网络更新这些提案的得分与框体坐标。

该方法将top-5定位误差降低至10.6%（表13）。这是我们在验证集上的单模型结果。采用网络集成方法进行分类与定位后，我们在测试集上实现了9.0%的 top-5定位误差。该数值显著优于ILSVRC 2014的结果（表14），实现了64%的相对误差降低。该结果在 *ILSVRC 2015的ImageNet*定位任务中获得第一名。